

ACID durability

- write ahead logging
- buffer management
- steal, no-force strategies
- checkpoints
- media recovery

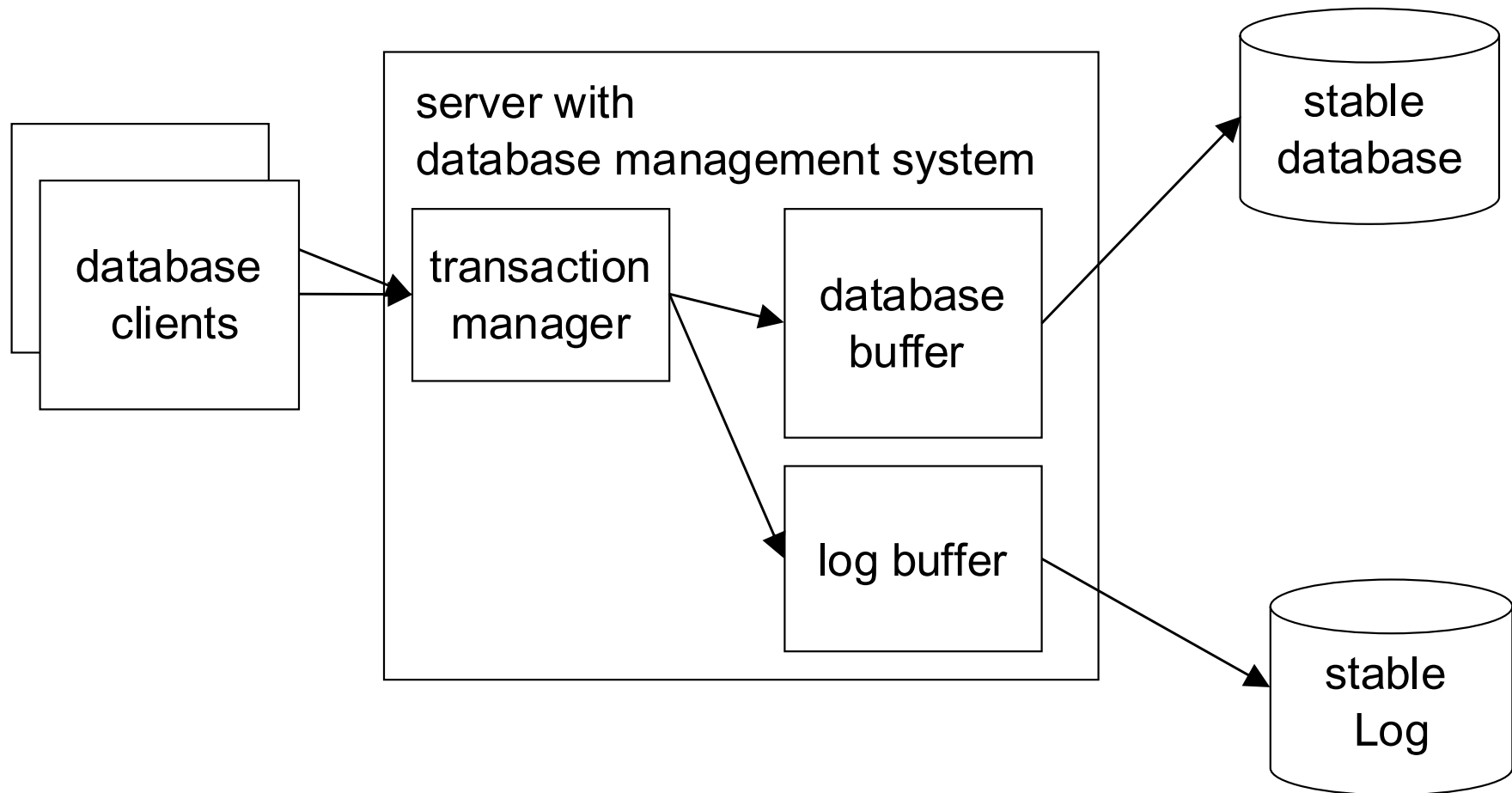
The ACID properties

- requirements that a transaction manager must meet for the transactions:
 - **Atomicity**: either all the operations of a transaction are made durable or none of them are.
 - **Consistency**: after the transaction, the database is in a consistent state.
 - **Isolation**: operations in a transaction appear isolated from all other operations. Transactions have a virtual serial view on the system.
 - **Durability**: once the user has been notified of success, the transaction will persist, and not be undone.

ACID durability

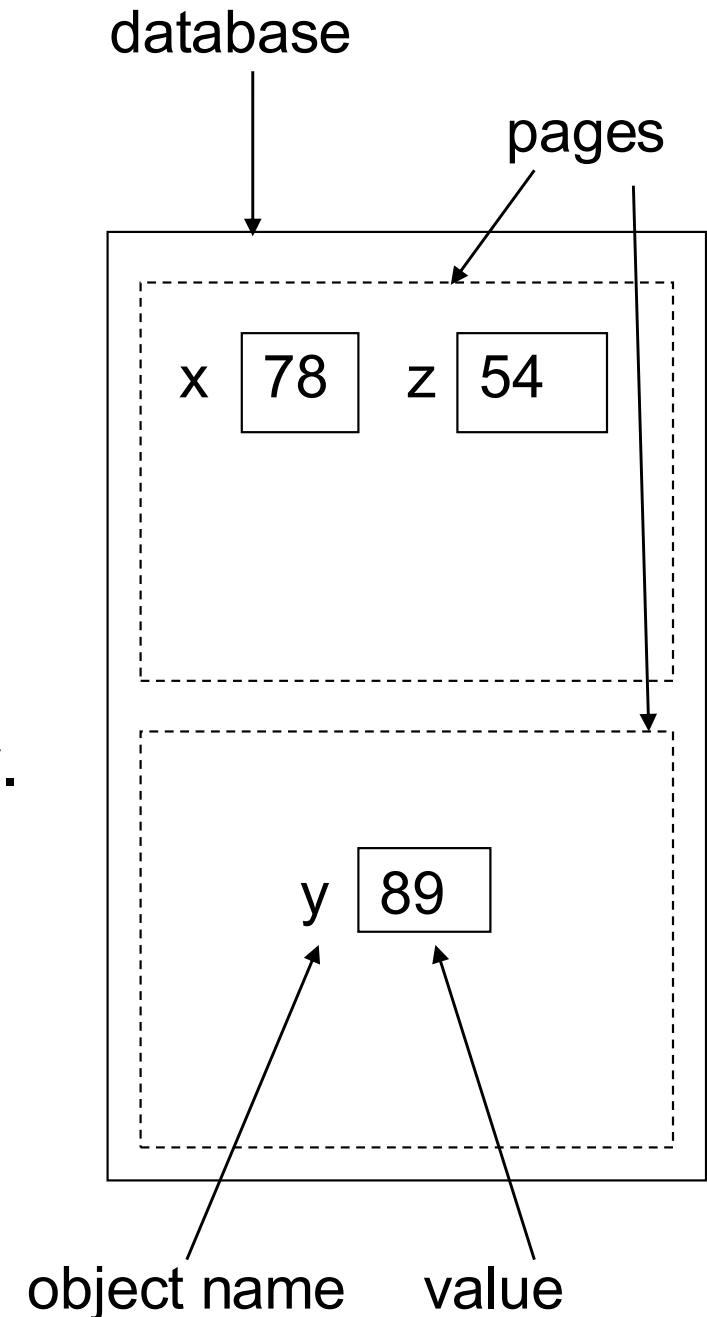
- Once the user has been notified of success of transaction t:
 - a) All writes of t are permanent: must be changed through later committed writes, e.g., compensating transactions.
 - b) The effect of t is kept in a crash resistant way.
 - minimum requirement: transaction is written to persistent storage: resistant against
 - OS crash
 - system outage
 - c) preferred: protection against loss of persistent memory:
 1. Resistance against persistent storage failure
 2. resistance against catastrophes, geographic distribution

System architecture



Structure of the database

- Database buffer and stable database content is partitioned into pages.
- Every buffer page has exactly one image page on the stable database.
- Complete pages are read from and written to the stable database on disk.
 - must be read when not yet in buffer.
- buffer is fast, stable database is vast.
- Difference in access time: RAM: 1ns, disk 1ms; ratio? SSD?



Crash recovery: write ahead logging 1

- A log has a *log buffer* in main memory and a *stable log* on persistent storage.
- Semantics of a **system crash**: At an arbitrary point in time, database buffer as well as log buffer are lost.
- Recovery must be based on stable log and stable database alone.
- A transaction is conceived as committed only *after* the commit entry of this transaction is written to the persistent and reliable log file storage: *write ahead logging (WAL)*.
 - Crucial part of durability.
- Main policy/semantics of database crash recovery:
The *stable log* is authoritative.

Crash recovery: stable log is authoritative

- The stable log decides about the correct status of transactions:
 - Committed transactions are exactly those that have a commit record in the stable log because of write ahead logging. They are *winners* and considered committed: crash-durability.
 - Other transactions (without a commit record in the stable log) are *losers* and considered aborted.

Goal of crash recovery: clean stable database

- The stable database is **clean** iff the stable database is consistent with the winner/loser decision of the stable log.
- All writes of the winners, and only the writes of the winners are reflected in the stable database.
- The log entries of uncommitted transactions must be without effect.
- Task of crash recovery: The stable database must be made clean based on the stable log.

Remark: media recovery

- Addresses a much more severe failure situation, but is semantically simple:
- Addresses the situation that the stable database is lost by media failure, i.e., Hard disk crash, catastrophes.
- Important semantic specification: The clean stable database can be reconstructed at any point in time:
 - from the complete stable log : Redo all transactions!
 - from a historic clean stable database copy and the stable log from that point in time: *media recovery*.
- Ergo: For media recovery, log must be independently durable on every commit! Backups for stable database!

Media recovery contd.

- Media recovery: takes place, if stable database is lost.
 - Remark: Loss of the log cannot be repaired
 - highly reliable store is used for log.
- Requires proper archiving:
 - Log is kept long-term, even after truncation.
 - All log entries are stored in a *log archive*.
 - From time to time, *database backups* are made from the stable database.

Database buffer management

- The buffer is a write cache: Changes to the data in the buffer are not immediately written to the stable database.
- Database buffer management:
 - if a cache miss occurs, load requested pages. This means, old pages must be replaced.
 - Pages with changes need to be written back.
 - We have to distinguish committed changes and uncommitted changes.

Crash recovery challenges

- Database buffer management can be aligned in different ways to transactions.
- Easiest situation: Stable database is always clean. Unfortunately, this will turn out to be not practical.
- Alternative, more complex situations:
 - The stable database can contain pages with the following problems:
 1. Old data not reflecting writes by committed transactions:
an **outdated (stale)** page.
 2. writes by uncommitted transactions: A **steal** page.
 - Both kinds of problems can appear on the same page.

Buffer management policy alternatives

- Policies for buffer pages with committed write.
 - **force**: At commit, such pages have to be written to the stable database. Leads to performance bottlenecks.
 - **no-force**: drops this requirement. Leads to redo.
- Policies for buffer pages with uncommitted write
 - **no-steal**: Such pages must not be written to stable database. Can lead to buffer bottlenecks.
 - **steal**: drops this requirement. Leads to undo.
- Force, no-steal is easy crash recovery, ensures the stable database is always clean: not practical enough.

Crash recovery challenges

- The stable database can contain pages with the following problems:
 1. Old data not reflecting writes by committed transactions:
an **outdated** (stale) page.
-> **no-force**
 2. writes by uncommitted transactions: A **steal** page.
-> **steal**
- Note: **no-steal** requires some kind of page lock: we need to get all open transactions off a page to write it back under no-steal policy. Page locks are a performance issue.

Buffer management: no-force, steal policy

- Algorithms for Recovery and Isolation Exploiting Semantics (ARIES) [Mohan et al. 1992]
- Is today's **preferred** solution: no alignment between buffer page swapping and transactions: *no-force, steal policy*:
 - **A full write cache delivering durability!**
- Buffer pages are swapped according to demand.
- Avoids more bottlenecks, more difficult to implement.
- **no-force**., some committed writes are not in the stable database yet. Makes redo after crash necessary.
- **steal**: some uncommitted writes are in the stable database: undo also after crash.

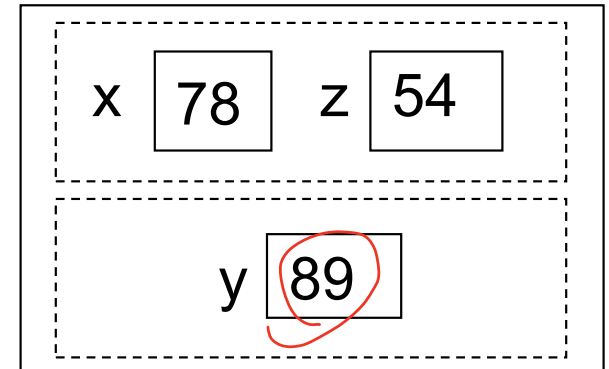
Write ahead logging 2 for steal

- enabling crash recovery for steal policy:
- stable database pages are changed by loser transactions.
- the information to undo the loser transactions must be in the log.
- Therefore,
 - before a buffer page is written back to the stable database:
 - all log entries for that page have to be written back to the stable log.
- This is another application of *write ahead logging*.

Recovery example, the scenario

- Example log using implicit transaction start, no BOT.
- When was the last time a page was written out.

Stable database



before Stable log *after*

[nr: 110, ta: 22, obj: x, b: 91, a: 78]

[nr: 111, ta: 23, obj: z, b: 23, a: 54]

[nr: 112, ta: 22, obj: x, b: 78, a: 53]

[nr: 113, ta: 22, obj: y, b: 89, a: 64]

[nr: 114, ta: 22, commit]

[nr: 116, ta: 23, obj: y, b: 64, a: 85]

[nr: 116, ta: 23, obj: z, b: 54, a: 37]

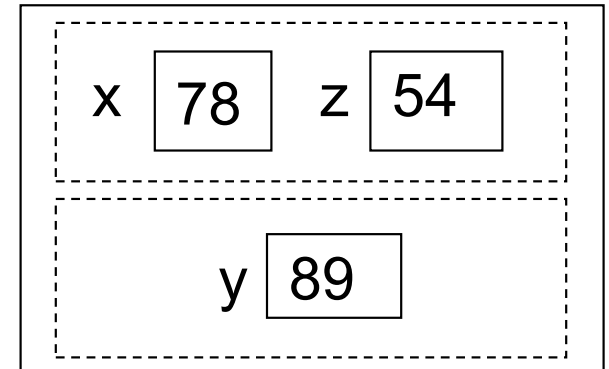
log buffer

... written out here

Recovery example, the scenario

- Situation at the time of the crash.
- Database buffer is then lost.
- Writes are shown on top of old values.

Stable database



Stable log

[nr: 110, ta: 22, obj: x, b: 91, a: 78]

[nr: 111, ta: 23, obj: z, b: 23, a: 54]

[nr: 112, ta: 22, obj: x, b: 78, a: 53]

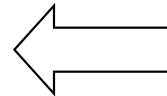
[nr: 113, ta: 22, obj: y, b: 89, a: 64]

[nr: 114, ta: 22, commit]

[nr: 116, ta: 23, obj: y, b: 64, a: 85]

[nr: 116, ta: 23, obj: z, b: 54, a: 37]

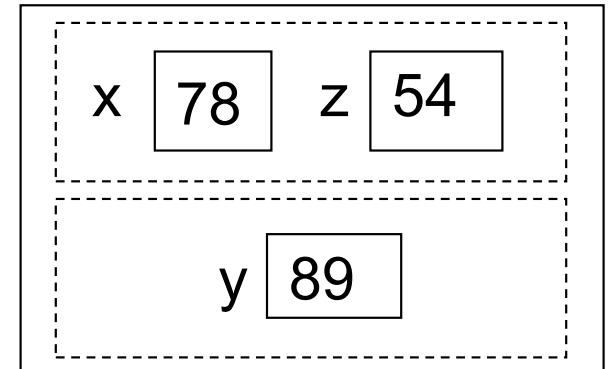
log buffer



Recovery example, the scenario

- What is the state of the database buffer at current time?

Stable database



Stable log

[nr: 110, ta: 22, obj: x, b: 91, a: 78]

[nr: 111, ta: 23, obj: z, b: 23, a: 54]

[nr: 112, ta: 22, obj: x, b: 78, a: 53]

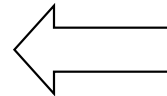
[nr: 113, ta: 22, obj: y, b: 89, a: 64]

[nr: 114, ta: 22, commit]

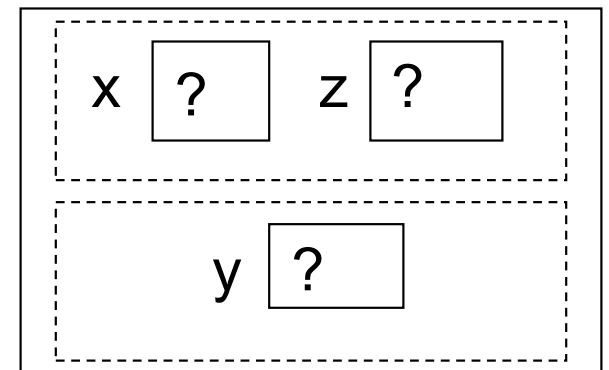
[nr: 116, ta: 23, obj: y, b: 64, a: 85]

[nr: 116, ta: 23, obj: z, b: 54, a: 37]

log buffer



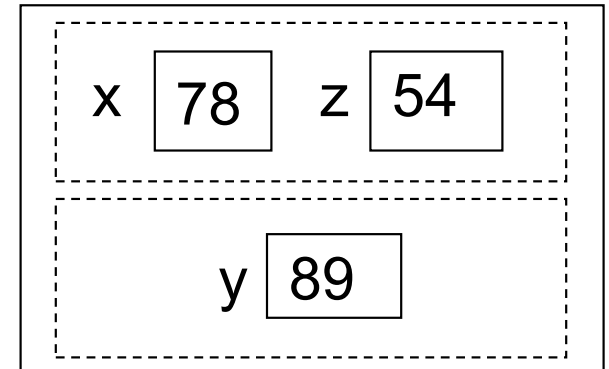
Database buffer



Recovery example, the scenario

- Situation at the time of the crash.
- Database buffer is then lost.
- What do we have to do?

Stable database



Stable log

[nr: 110, ta: 22, obj: x, b: 91, a: 78]

[nr: 111, ta: 23, obj: z, b: 23, a: 54]

[nr: 112, ta: 22, obj: x, b: 78, a: 53]

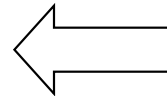
[nr: 113, ta: 22, obj: y, b: 89, a: 64]

[nr: 114, ta: 22, commit]

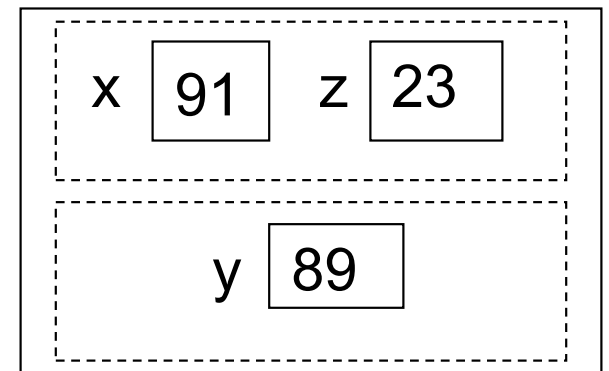
[nr: 116, ta: 23, obj: y, b: 64, a: 85]

[nr: 116, ta: 23, obj: z, b: 54, a: 37]

log buffer



Database buffer



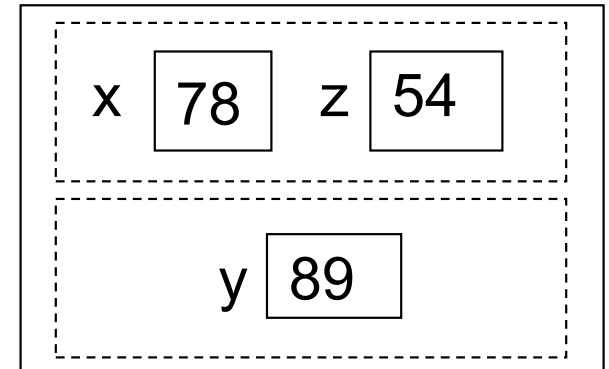
Crash recovery for no-force, steal policy

- has to redo winners and undo losers.
- has to go through the log:
- for redo in positive time direction
 - Identify, whether the write (or its TA) is committed.
 - write for each committed operation the after-image.
- for undo in negative time direction
 - Identify whether write (or its TA) is not committed
 - write for each uncommitted operation the before-image.

Recovery example, the scenario

- Situation at the time of the crash.
- Database buffer is then lost.
- Writes are shown on top of old values.

Stable database



uncommit.

Stable log

[nr: 110, ta: 22, obj: x, b: 91, a: 78]

[nr: 111, ta: 23, obj: z, b: 23, a: 54]

[nr: 112, ta: 22, obj: x, b: 78, a: 53]

[nr: 113, ta: 22, obj: y, b: 89, a: 64]

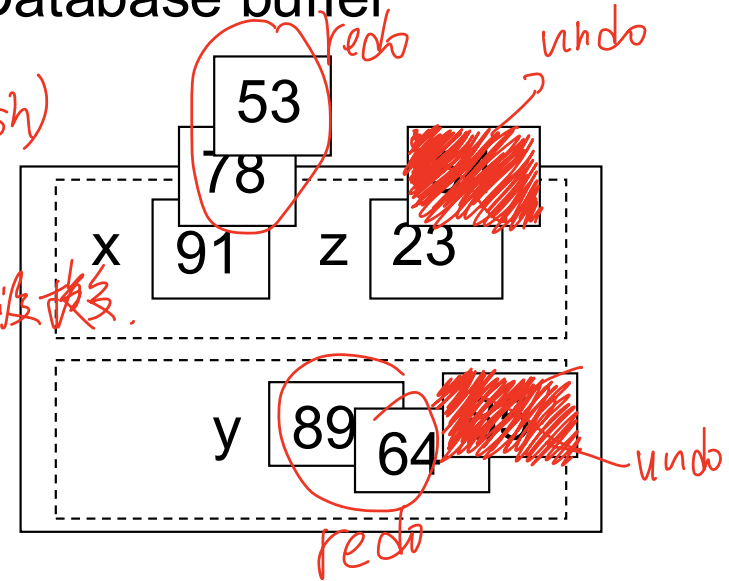
[nr: 114, ta: 22, commit]

[nr: 116, ta: 23, obj: y, b: 64, a: 85]

[nr: 116, ta: 23, obj: z, b: 54, a: 37]

log buffer (lost)

Database buffer



(crash)

只提交了22, 23没提交.

不一致.

Summary

- ACID Atomicity and ACID Durability can be achieved with strategies working with an undo/redo log.
- The stable log and the stable database reside on persistent memory (disks, SSD).
- System crash: main memory content is lost.
- write-ahead logging: the stable log is authoritative, can be used to reconstruct a clean stable database.
- Different managements of the database buffer are possible, with the alternatives force/no-force, steal/no-steal.
- In the steal, no-force strategy, we have to redo winners and undo loser transactions.